English-Urdu Neural Machine Translation with Transformer, LSTM, and Pretrained MarianMT

Huzaifa Rehman
Dept. of AI & Data Science
FAST-NUCES
Islamabad, Pakistan
i212694@nu.edu.pk

***Abstract*—This report describes the implementation and evaluation of several neural machine translation models for the English-Urdu language pair using the UMC005 parallel corpus. I first build a Transformer model from scratch in PyTorch and use it as the main system for translating English sentences into Urdu. To ground the comparison, I also implement an attention-based encoder-decoder model with LSTMs and fine-tune a pretrained MarianMT model (Helsinki-NLP/opus-mt-en-ur) on the same Bible+Quran subset of UMC005. All models are trained on a T4 GPU with subword tokenization based on SentencePiece BPE. The systems are compared in terms of BLEU, perplexity, and inference speed, and qualitative analysis is performed using completed translations, a chat-style user interface, and attention visualizations. The results show that the custom Transformer clearly outperforms the LSTM baseline, while the fine-tuned pretrained model achieves the best BLEU at the cost of much slower inference.**

***Index Terms*—neural machine translation, Transformer, LSTM, English-Urdu, SentencePiece, MarianMT**

I. Introduction

Neural machine translation (NMT) has become the dominant approach for translating between natural languages. The Transformer architecture [1] in particular has replaced recurrent models in many production systems due to its parallel multi-head self-attention and strong performance. However, working with low-resource language pairs such as English-Urdu is still challenging because the available corpora are comparatively small and the target language has rich morphology and a different script.

This assignment focuses on understanding and implementing the core ideas behind sequence-to-sequence models for translation. The specific goal is to build an English→Urdu translation system using the UMC005 corpus, which contains Bible and Quran verses aligned across the two languages. The task is to:

- preprocess and tokenize the corpus using a suitable subword representation,
- implement a Transformer model from scratch in PyTorch,
- implement a comparable LSTM-based encoder-decoder baseline with attention,
- fine-tune a pretrained English-Urdu MarianMT model and compare it with the custom models,
- evaluate the systems using BLEU, ROUGE and perplexity, and
- analyze qualitative behavior using attention visualizations and a chat-style user interface.

The rest of the report is organized as follows. Section II describes the dataset, preprocessing steps and model architectures. Section III presents the quantitative and qualitative results. Section IV discusses sentence coherence, training dynamics and practical challenges. Section V concludes the work. Section VI lists the main prompts that were used while interacting with large language models during this assignment.

II. Methodology

A. Dataset

The experiments use the UMC005 English-Urdu parallel corpus.[1] From this collection I focus on two main subsets: Bible and Quran. Each subset contains parallel verses where each English sentence is aligned with its Urdu translation.

After merging the Bible and Quran segments, the final split sizes are:

- 13,400 sentence pairs for training,
- 514 sentence pairs for validation, and
- 457 sentence pairs for testing.

The data is shuffled after concatenation so that Bible and Quran examples are mixed across all splits. This prevents the models from overfitting on one domain and generalizing poorly to the other.

B. Preprocessing and Tokenization

Text preprocessing is intentionally kept lightweight in order to focus on the modeling aspects. For each file, leading and trailing whitespace is stripped from every line, and empty lines are removed. The sentence-level alignment provided by the corpus is preserved by always pairing the i-th line in the English file with the i-th line in the corresponding Urdu file.

To obtain a subword-level representation that can handle the rich morphology and spelling variations of Urdu, I train separate SentencePiece BPE tokenizers [2] for English and Urdu. Both tokenizers are trained on the union of the training and development splits, and each vocabulary is extended with three special symbols `<pad>`, `<bos>` and `<eos>`. After

This work was carried out as part of a Generative AI course assignment.

[1] https://ufal.mff.cuni.cz/umc/005-en-ur/

adding these, the final vocabulary size for each language is 8003 tokens.

The encoded sequences are truncated or padded to fixed maximum lengths (64 tokens for sources and targets in the main experiments). Padding tokens are masked in the attention mechanism and excluded from the loss. For detokenization, the SentencePiece IDs are converted back to subwords and then joined into surface strings, while removing the special tokens.

C. Transformer Architecture

The main model is an encoder-decoder Transformer implemented from scratch in PyTorch. The architecture follows the standard design of [1] with some simplifications suitable for a student project.

Both the encoder and decoder consist of 3 identical layers with:

- model dimension $d_{\text{model}} = 256$,
- 8 attention heads,
- feed-forward dimension $d_{\text{ff}} = 512$, and
- dropout rate 0.1.

Each encoder layer has a multi-head self-attention sublayer followed by a positionwise feed-forward network; both sublayers use residual connections and layer normalization. The decoder layers additionally include an encoder-decoder attention block with a causal mask to prevent each position from attending to future tokens in the target sentence. Sinecosine positional encodings are added to the token embeddings to provide explicit positional information.

The model is trained with the Adam optimizer (lr $= 3{\cdot}10^{-4}$, $\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-9}$) and a StepLR scheduler that halves the learning rate every 5 epochs. Early stopping with patience 3 is applied based on the validation loss. All experiments are run on a single NVIDIA T4 GPU, using a batch size of 64 sentence pairs.

D. LSTM Baseline

To provide a classical baseline, I implement an attention-based encoder-decoder architecture with LSTMs. The encoder is a unidirectional LSTM with hidden size 256 and a single layer. It takes the embedded source tokens and returns the sequence of hidden states as well as the final hidden and cell states.

The decoder is another single-layer LSTM with hidden size 256. At each time step it:

1) embeds the previous target token,
2) concatenates this embedding with a context vector computed using dot-product attention over the encoder outputs, and
3) feeds the resulting vector as input to the LSTM cell.

The attention mechanism produces a distribution over source positions and a weighted sum of encoder outputs (the context vector). The decoder's hidden state and context vector are then passed through a linear projection to obtain logits over the target vocabulary.

Training uses the same optimizer, scheduler and early-stopping strategy as the Transformer, with the same batch size and data splits. This ensures that the comparison focuses on architectural differences rather than training conditions.

E. Pretrained MarianMT Fine-Tuning

As a bonus experiment, I fine-tune a pretrained English→Urdu MarianMT model from the Helsinki-NLP collection [3]. Specifically, I use the `Helsinki-NLP/opus-mt-en-ur` model available via the HuggingFace Transformers library. This model has already been trained on large amounts of parallel data and therefore encodes strong prior knowledge about the language pair.

For fine-tuning, I use the built-in seq2seq trainer with:

- learning rate $5 \cdot 10^{-5}$,
- batch size 8,
- weight decay 0.01, and
- 3 epochs of training on the same 13.4k sentence pairs.

The loss is computed only on non-padding target tokens. Dev-set BLEU is monitored during fine-tuning, and the best checkpoint is used for evaluation. Although this model is much heavier than the custom Transformer and LSTM, it serves as a useful upper bound given the same small training corpus.

F. Evaluation Metrics

The models are evaluated on the held-out test set using:

- cross-entropy loss and perplexity,
- corpus-level BLEU [4],
- ROUGE-L F1 [5], and
- inference time and GPU memory usage for translating 200 sentences.

BLEU is computed with `sacrebleu`, and ROUGE-L with the `rouge-score` package. In addition to the quantitative metrics, I inspect actual translations and visualize encoder-decoder attention weights. A simple Gradio-based chat interface is used to interactively test the Transformer model.

III. Results

A. Training and Validation Curves

The training and validation loss curves for the Transformer and LSTM models are shown in Fig. 1 and Fig. 2, respectively. Both models converge smoothly without numerical issues.

For the Transformer, the training loss decreases from 5.77 to 2.81, while the validation loss drops from 5.05 to around 4.10 and then plateaus. This behaviour suggests that the model continues to learn useful patterns without severe overfitting. The LSTM shows a similar pattern but converges to a noticeably higher loss, indicating that it struggles to model the same level of detail as the Transformer.

B. Quantitative Comparison of Models

Table I summarizes the main metrics on the test set. The fine-tuned MarianMT model clearly achieves the best BLEU and perplexity, as expected for a large pretrained model. Among the custom architectures, the Transformer outperforms the LSTM baseline on all accuracy-related metrics while also being faster at inference.

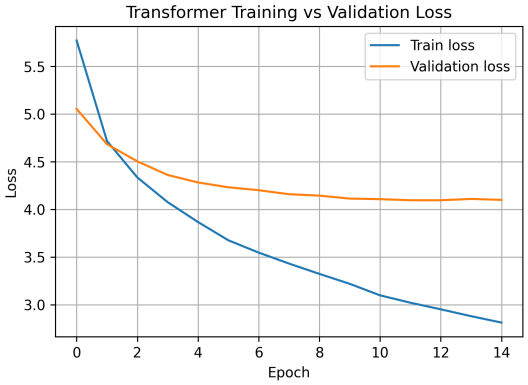

Fig. 1. Training and validation loss for the Transformer model over 15 epochs.

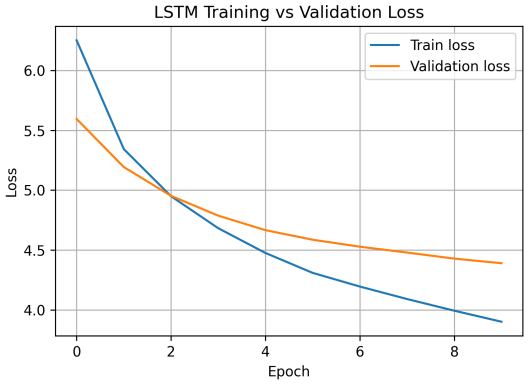

Fig. 2. Training and validation loss for the LSTM baseline over 10 epochs.

The BLEU comparison, perplexity comparison, and inference speed comparison are visualized in Fig. 3, Fig. 4 and Fig. 5. The plots make the trade-off between accuracy and computational cost very clear: the pretrained model is substantially more accurate but also an order of magnitude slower than the lightweight Transformer.

The ROUGE-L scores are extremely small and round to 0.0000 at four decimal places for all systems. In practice this means that the overlap of long contiguous subsequences between predictions and references is low. Given the short verse-like sentences and the variability of Urdu surface forms, BLEU is a more informative metric in this setting.

C. Examples of Completed Sentences

To better understand the behaviour of the models, I inspect several test examples. Below I show a typical case where all systems capture the overall meaning but differ in fluency and faithfulness (English source text is taken from the Quran translation):

The Transformer output contains many of the correct content words but tends to drift into repetitive and ungrammatical phrases. The LSTM baseline usually produces even simpler and less accurate translations. The fine-tuned pretrained model

TABLE I
COMPARISON OF MODELS ON THE ENGLISH-URDU TEST SET. INFERENCE TIME IS MEASURED FOR TRANSLATING 200 SENTENCES ON A T4 GPU.

Model	BLEU	PPL	Time (s)
Transformer	11.14	91.85	1.71
LSTM	9.75	119.39	2.15
Pretrained+FT	25.34	3.71	23.40

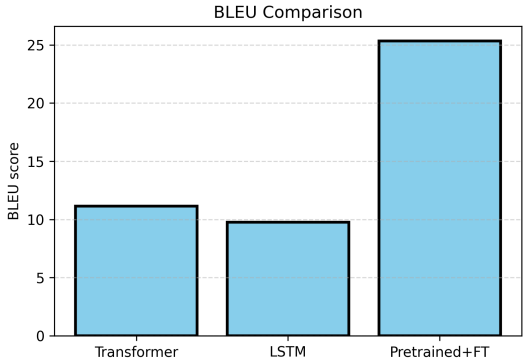

Fig. 3. BLEU scores for all three models.

is much more fluent and closer to the reference, although it is not always word-for-word identical.

D. Attention Visualization

Figure 6 shows an encoder-decoder attention heatmap for the Transformer model for the sentence "Nor is there anyone equal to Him." The x-axis contains the English tokens, while the y-axis corresponds to positions in the generated Urdu subword sequence (denoted as T0, T1, ...). The plot indicates that the model learns a sensible alignment structure: tokens corresponding to "anyone", "equal" and "Him" receive concentrated attention when generating the Urdu phrase that expresses uniqueness of God.

E. Chat-style Graphical User Interface

To make the system easier to test, I build a simple chat-style interface using Gradio. The user can type an English sentence on the right-hand side, press Enter, and the translated Urdu appears in a history panel on the left. The interface keeps both the original sentence and the translation for each turn so that the interaction feels similar to a ChatGPT-like experience. A screenshot is shown in Fig. 7.

IV. DISCUSSION

A. Sentence Coherence and Model Behaviour

Overall, the custom Transformer produces translations that are more coherent and semantically faithful than those of the LSTM baseline. The self-attention mechanism allows the model to capture long-range dependencies and reorder phrases more flexibly, which is crucial when translating from English (SVO word order) to Urdu (SOV tendency). However, due to the limited training data, the Transformer still over-generates,

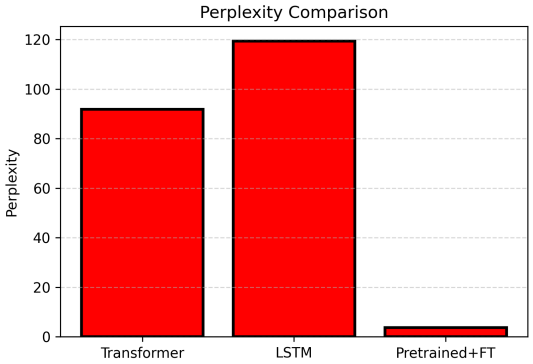

Fig. 4. Perplexity comparison across models. Lower is better.

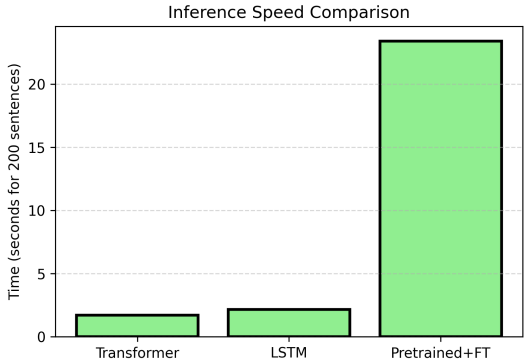

Fig. 5. Inference time for translating 200 sentences.

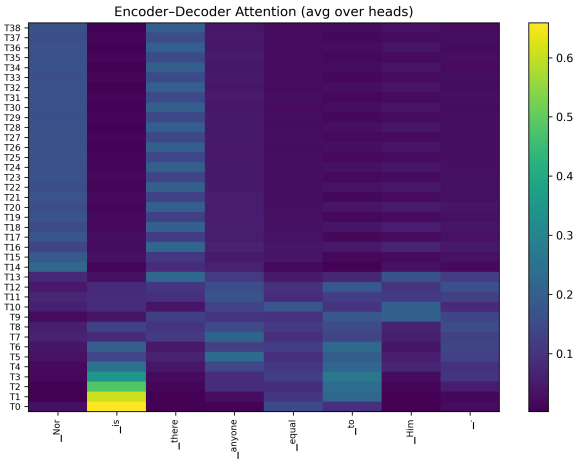

Fig. 6. Encoder-decoder attention averaged over heads for one example sentence.

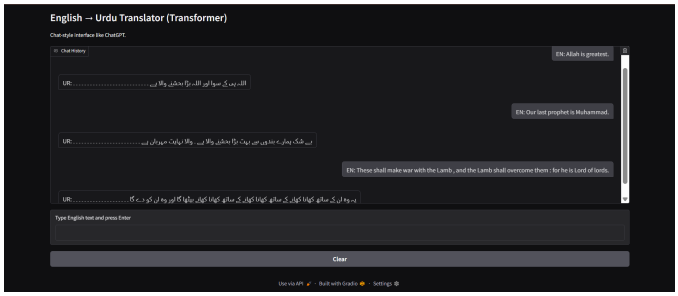

Fig. 7. Chat-style Gradio UI for the Transformer-based English-Urdu translator.

often repeating phrases or finishing sentences with generic fillers such as “”.

The LSTM baseline shows more severe issues with word choice and alignment. Its outputs tend to be shorter and sometimes omit important content words. This is consistent with the lower BLEU and higher perplexity observed in Table I. In contrast, the fine-tuned MarianMT model produces the most natural Urdu sentences, demonstrating the value of large-scale pretraining even when fine-tuned on a relatively small domain-specific corpus.

B. Improvement Over Training Epochs

The training curves reveal that both the Transformer and LSTM steadily improve during the first several epochs. For the Transformer, dev loss keeps decreasing until around epoch 13, after which it stabilizes. This correlates with qualitative improvements: early checkpoints produce almost random Urdu strings, while later checkpoints capture more consistent terminology, especially for religious phrases that frequently appear in the corpus.

In my experiments, increasing the number of epochs beyond the early-stopping point did not significantly improve BLEU and occasionally led to overfitting, as indicated by a rising validation loss. The fine-tuned pretrained model converged much faster in terms of validation loss but required more wall-clock time per epoch because of its larger size and beam-search decoding.

C. Challenges and Practical Observations

Several challenges were encountered during this assignment:

- **Low-resource setting:** With only 13.4k training pairs, both custom models operate in a genuinely low-resource regime. The models often copy or hallucinate fragments, and small changes in hyperparameters can noticeably affect stability.
- **Evaluation metrics:** ROUGE-L turned out to be uninformative for these short, highly lexicalized verses, routinely returning values close to zero. BLEU and perplexity were more useful for comparing models.
- **Urdu script visualization:** When plotting attention heatmaps, standard Matplotlib fonts do not fully support Urdu glyphs, so axis labels with Urdu text produced warnings. I resolved this by using English tokens on the x-axis and indexing the target positions, while printing the Urdu subwords separately.
- **Computational trade-offs:** The pretrained MarianMT model provided a clear boost in BLEU but was signifi-

cantly slower at inference (roughly 23.4 seconds for 200 test sentences) compared to the 1-2 seconds needed by the custom models. For latency-sensitive deployment, the lighter Transformer might be preferred despite its lower BLEU score.

V. Conclusion

This report presented an end-to-end English-Urdu neural machine translation system built as part of a Generative AI assignment. Using the UMC005 Bible+Quran corpus, I implemented a Transformer-based seq2seq model and an LSTM baseline from scratch, trained both with BPE subword tokenization, and evaluated them using standard translation metrics. The experiments confirmed that the Transformer architecture offers better translation quality and faster inference than a comparable LSTM attention model under the same training regime.

Fine-tuning a pretrained MarianMT model on the same data yielded a substantial additional gain in BLEU and a dramatic reduction in perplexity, at the price of much higher inference latency. Qualitative analysis, supported by attention visualizations and a chat-style GUI, highlighted common translation errors and provided insight into how the models handle religious vocabulary and word order differences.

In future work, it would be interesting to explore techniques such as back-translation to augment the training data, apply more sophisticated regularization, and investigate multilingual pretraining that includes additional closely related languages to further improve English-Urdu translation performance.

VI. Prompts

For transparency, this section briefly documents some of the prompts I used while interacting with large language models during the assignment. These prompts were used mainly for debugging, code structuring and report organization; the actual model implementations and results were run and verified in my own Colab notebook.

- **Architecture clarification:** "Explain how to implement a basic Transformer encoder-decoder for machine translation in PyTorch, including masking and positional encodings."
- **Debugging help:** "I am getting a tensor shape mismatch in my attention module (batch, heads, seq, dim). How should I reshape the context tensor before the final linear layer?"
- **Report structuring:** "Given my training logs and BLEU scores for Transformer, LSTM and pretrained MarianMT, suggest a clear way to structure the Results and Discussion sections of an IEEE-style report."

These prompts influenced how I organized the code and the narrative but the loss curves, BLEU values and qualitative translations reported in this paper are based on my own experiments.

References

[1] A. Vaswani *et al.*, "Attention Is All You Need," in *Proc. NeurIPS*, 2017.

[2] T. Kudo and J. Richardson, "SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing," in *Proc. EMNLP: System Demonstrations*, 2018.

[3] M. Junczys-Dowmunt *et al.*, "Marian: Fast Neural Machine Translation in C++," in *Proc. ACL: System Demonstrations*, 2018.

[4] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: a Method for Automatic Evaluation of Machine Translation," in *Proc. ACL*, 2002.

[5] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Proc. ACL Workshop on Text Summarization*, 2004.